

WiseMouth — 48-Hour Hackathon Plan

Product

WiseMouth

Tagline

See what the conversation doesn't explicitly say.

One-liner

WiseMouth is a real-time conversation intelligence system that listens to conversations through AssemblyAI and identifies observable conversational signals such as uncertainty, hedging, commitments, changing commitments, repeated concerns, disagreements, and unanswered questions.

Core principle

WiseMouth does **not** claim to read emotions, detect lies, or know what someone is thinking.

It analyzes observable language and conversation patterns.

> **AssemblyAI hears it. Rules detect it. The LLM understands it. Temporal intelligence connects it. WiseMouth visualizes it.**

1. Problem

Normal meeting and conversation tools mostly produce:

- Transcripts
- Summaries
- Action items

But important information can be hidden in the progression of the conversation:

- Someone agrees but uses hesitant language.
- Someone makes a commitment and later weakens it.
- The same concern is raised repeatedly.
- Two people disagree without reaching a decision.
- A question is asked and nobody answers.

- A requirement changes during the discussion.

WiseMouth makes these observable conversational patterns visible in real time.

2. Core Experience

Example:

Sarah:

> "Yeah, I guess we could deploy Friday."

WiseMouth:

HEDGING / UNCERTAINTY DETECTED

> "I guess we could..."

>

> Signal: Low-commitment language.

Later:

John:

> "I'll handle the deployment."

WiseMouth:

COMMITMENT DETECTED

Then later:

John:

> "Actually, I don't think I'll have enough time to do that."

WiseMouth:

COMMITMENT CHANGE

Earlier:

> "I'll handle the deployment."

Later:

> "I don't think I'll have enough time."

This temporal connection is one of the product's primary differentiators.

3. Signal Categories

3.1 Uncertainty

Detect language such as:

- "I'm not sure."
- "Maybe."
- "Probably."
- "I guess."
- "I think."
- "It should be okay."

Output:

UNCERTAINTY

> Sarah expressed uncertainty about the deployment date.

3.2 Hedging

Detect language that technically agrees or proposes something but weakens commitment.

Examples:

- "That should probably work."
- "I suppose that's fine."
- "We could potentially do that."

Output:

HEDGED AGREEMENT

> Agreement detected, but commitment strength appears low based on language.

3.3 Commitment

Detect explicit commitments.

Examples:

- "I'll handle it."
- "I'll finish the API today."
- "I'll prepare the deployment checklist."

Output:

COMMITMENT

- Owner: John
- Action: Prepare deployment checklist
- Timestamp: 00:08:12

3.4 Commitment Change

Compare a person's current statement with an earlier commitment.

Example:

Earlier:

> "I'll finish the API today."

Later:

> "I don't think I'll be able to finish it."

Output:

COMMITMENT CHANGED

Store both statements and timestamps as evidence.

3.5 Repeated Concern

Detect the same concern appearing multiple times.

Example:

Maria:

> "I'm worried about the database migration."

Later:

> "But the migration could cause downtime."

Later:

> "I still don't think we're ready."

Output:

REPEATED CONCERN

> Maria raised concerns about database migration three times.

>

> Topic remains unresolved.

3.6 Unresolved Disagreement

Example:

Michael:

> "We should use PostgreSQL."

Maria:

> "I think MongoDB makes more sense."

Michael:

> "I still prefer PostgreSQL."

No decision follows.

Output:

UNRESOLVED DISAGREEMENT

- Topic: PostgreSQL vs MongoDB
- Participants: Michael, Maria
- Status: No final decision detected

3.7 Unanswered Question

Example:

Maria:

> "Who's going to handle the production migration?"

Nobody answers.

The conversation moves on.

Output:

UNANSWERED QUESTION

> "Who's going to handle the production migration?"

>

> No response detected.

4. Live Dashboard

The main screen should be a two-panel interface.

Left: Live Transcript

Show:

- Speaker
- Transcript
- Timestamp
- Live recording indicator

Right: Conversation Radar

Example:

CONVERSATION RADAR

Uncertainty 4
Hedging 2
Commitments 5
Commitment Changes 1
Repeated Concerns 2
Disagreements 1
Unanswered Questions 2

Below the counters:

LIVE SIGNAL

COMMITMENT CHANGED

John previously committed
to handling deployment.

Earlier:
"I'll handle the deployment."

Now:
"I don't think I'll have time."

The UI should feel alive and update immediately.

5. Conversation Timeline

After a conversation:

00:04 Uncertainty
00:08 Commitment
00:13 Disagreement
00:17 Repeated Concern
00:24 Commitment Changed
00:31 Unanswered Question

Clicking a signal opens its evidence.

Every important signal should contain:

- Signal type
- Speaker
- Evidence text
- Timestamp
- Related earlier statement when applicable
- Confidence
- Topic/context

6. Evidence-First Design

WiseMouth should never simply say:

> "John became less confident."

Instead:

Commitment Change

Earlier — 10:14

> "I'll finish the API today."

Later — 10:47

> "I don't think I'll be able to finish it."

Detected pattern:

Commitment → weakened commitment

Confidence: High

The evidence makes the system transparent and defensible.

7. Product Boundaries

WiseMouth should NOT claim:

- Emotion detection
- Lie detection
- Personality profiling
- Psychological diagnosis
- "Sarah is angry"
- "John is lying"
- "Maria secretly disagrees"

Instead, use observable descriptions:

- Uncertainty language detected

- Hedging detected
- Commitment detected
- Commitment changed
- Repeated concern detected
- Unresolved disagreement detected
- Unanswered question detected

8. Technology Stack — Free First

Frontend

- Next.js
- TypeScript
- Tailwind CSS
- shadcn/ui
- Lucide Icons
- Recharts

Backend

- Python
- FastAPI
- WebSockets
- Pydantic
- SQLAlchemy

Speech

- AssemblyAI Realtime Speech-to-Text

AssemblyAI is the core realtime speech layer.

AI

Primary:

- Groq API / free allowance

Fallback:

- Gemini API / free allowance

Optional emergency fallback:

- Ollama with a local model

Do not depend on local inference for the primary demo.

Database

- PostgreSQL

Development can use Docker PostgreSQL.

Hosted PostgreSQL can use a free-tier provider if needed.

Deployment

Frontend:

- Vercel

Backend:

- Render or Oracle Cloud

Use whichever deployment route is more reliable during the hackathon.

Development

- GitHub
- Docker
- VS Code
- Claude Code

9. Final Architecture

```

MICROPHONE
|
v
+-----+
| Next.js |
| WiseMouth UI |
+-----+
|
WebSocket
|
v
+-----+
| FastAPI |
| Realtime GW |
+-----+
|
v
+-----+
| AssemblyAI |
| Realtime STT |
+-----+
|
v
Transcript
|
v
+-----+
| Signal Engine |
+-----+
|
+-----+
| |
v v
Rule Detection LLM Analysis
| |
+-----+
|
v
Conversation State
|
v
PostgreSQL
|
v
WiseMouth UI

```

10. Hybrid Signal Engine

Do not send every transcript segment directly to the LLM.

Use a hybrid architecture.

```

Transcript Chunk
|
v
Rule Detector
|
+---- No potential signal ----> Ignore
|
+---- Potential signal -----> LLM
|
v
Structured Signal
|
v
Conversation State

```

Benefits:

- 1 Lower LLM usage
- 2 Faster detection
- 3 Lower cost
- 4 More deterministic behavior
- 5 Easier debugging

11. Rule-Based Candidate Detection

Potential uncertainty/hedging terms:

- maybe
- probably
- I guess
- I think
- not sure
- might
- could
- perhaps

Potential commitment phrases:

- I'll
- I will
- I can
- I'll handle
- I'll take care of
- I'll finish
- I'll prepare

Potential questions:

- who
- what
- when
- where

- why
- how
- question marks
- interrogative structures

These rules do not make the final judgment. They identify candidate transcript segments that deserve deeper analysis.

12. LLM Structured Output

The LLM should return structured JSON rather than free-form prose.

Example:

```
{
  "signals": [
    {
      "type": "uncertainty",
      "speaker": "Sarah",
      "evidence": "I guess Friday should work",
      "timestamp": "00:04:32",
      "confidence": 0.91
    },
    {
      "type": "commitment",
      "speaker": "John",
      "action": "Handle deployment",
      "timestamp": "00:08:12",
      "confidence": 0.96
    }
  ]
}
```

Commitment change:

```
{
  "type": "commitment_change",
  "speaker": "John",
  "previous": {
    "text": "I'll handle the deployment.",
    "timestamp": "00:08:12"
  },
  "current": {
    "text": "I don't think I'll have time.",
    "timestamp": "00:24:41"
  }
}
```

13. Conversation State

The backend maintains state throughout the conversation.

Example:

```

{
  "commitments": [],
  "questions": [],
  "concerns": [],
  "disagreements": [],
  "topics": [],
  "signals": []
}

```

When a new transcript segment arrives:

```

New transcript
|
v
Signal extraction
|
v
Compare with existing state
|
v
New signal?
|
v
Update conversation state
|
v
Push update to frontend

```

14. Temporal Intelligence

This is a key differentiator.

Example state:

```

John
|
+-- "I'll deploy Friday."
| 10:14
|
+-- "I don't think I'll have time."
10:47

```

The system should compare new statements against previous commitments.

Process:

```

New statement
|
v
Existing commitments?
|
v
Same speaker?
|
v
Same topic?
|
v
Conflict or weakening?
|
v
COMMITMENT_CHANGED

```

Use the LLM for semantic matching when simple rules are insufficient.

15. Database Schema

Keep the database small.

conversations

- id
- title
- started_at
- ended_at
- status

transcript_segments

- id
- conversation_id
- speaker
- text
- timestamp

signals

- id
- conversation_id
- type
- speaker
- evidence
- timestamp
- confidence
- metadata

commitments

- id

- conversation_id
- speaker
- commitment
- status
- original_timestamp
- changed_timestamp

questions

- id
- conversation_id
- question
- speaker
- answered
- timestamp

No vector database is required for the MVP.

16. API / Realtime Concept

Frontend connects to FastAPI using WebSockets.

Example transcript event:

```
{
  "type": "transcript",
  "speaker": "John",
  "text": "I'll handle the deployment."
}
```

Example signal event:

```
{
  "type": "signal",
  "signal": {
    "type": "commitment",
    "speaker": "John",
    "confidence": 0.96
  }
}
```

The frontend immediately updates the Radar.

17. Suggested Repository Structure

```
wisemouth/  
|  
+-- frontend/  
| +-- app/  
| +-- components/  
| | +-- radar/  
| | +-- transcript/  
| | +-- timeline/  
| | +-- ui/  
| +-- hooks/  
| +-- lib/  
|  
+-- backend/  
| +-- app/  
| +-- api/  
| +-- services/  
| | +-- assemblyai.py  
| | +-- llm.py  
| | +-- signal_engine.py  
| | +-- conversation_state.py  
| +-- models/  
| +-- schemas/  
| +-- main.py  
|  
+-- tests/  
|  
+-- docker-compose.yml  
+-- .env.example  
+-- README.md
```

18. Environment Variables

```
ASSEMBLYAI_API_KEY=  
  
LLM_PROVIDER=groq  
  
GROQ_API_KEY=  
GEMINI_API_KEY=  
  
DATABASE_URL=  
  
NEXT_PUBLIC_API_URL=
```

Do not commit secrets.

19. MVP Screens

Screen 1 — Landing

```
WISEMOUTH  
  
See what the conversation  
doesn't explicitly say.  
  
[ Start Conversation ]
```

Screen 2 — Live Conversation

Two columns:

- Live transcript
- Conversation Radar

Screen 3 — Signal Details

Show evidence and timestamps.

Screen 4 — Conversation Report

Show:

- Total signals
- Commitments
- Commitment changes
- Uncertainty
- Hedging
- Repeated concerns
- Disagreements
- Unanswered questions

Screen 5 — Timeline

Visual chronological signal map.

20. 48-Hour Build Priority

Hours 0–4

Foundation:

- Repository
- Next.js
- FastAPI

- Docker
- PostgreSQL
- WebSocket skeleton
- Environment configuration

Hours 4–10

AssemblyAI:

- Microphone capture
- Realtime connection
- Live transcription
- Speaker handling
- Transcript persistence

Do not move forward until this works reliably.

Hours 10–16

Signal Engine:

- Rule-based candidate detection
- LLM structured extraction
- Signal schema
- Conversation state

Hours 16–22

Live Radar:

- Signal counters
- Live signal cards
- Evidence display
- Transcript synchronization

Hours 22–28

Temporal Intelligence:

- Commitment tracking
- Commitment changes
- Repeated concerns
- Unresolved questions
- Disagreements

Hours 28–34

Report:

- Signal summary
- Timeline
- Evidence view
- Conversation history

Hours 34–40

Polish:

- Animations
- Empty states
- Loading states
- Error handling
- Responsive layout
- Demo data fallback

Hours 40–44

Demo reliability:

- Deterministic demo conversation
- Test microphone
- Test AssemblyAI
- Test LLM fallback
- Test network failures

Hours 44–48

Presentation:

- Final UI polish
- Demo rehearsal
- Pitch
- Architecture slide
- Backup recording/demo
- No major new features

21. Demo Scenario

Use a controlled project-planning conversation.

Step 1 — Commitment

John:

> "I'll handle the deployment."

WiseMouth:

COMMITMENT DETECTED

Step 2 — Uncertainty

Sarah:

> "I'm not sure we're actually ready for Friday."

WiseMouth:

UNCERTAINTY DETECTED

Step 3 — Repeated Concern

Maria:

> "I'm still concerned about authentication."

Later:

> "The authentication service is still my biggest concern."

WiseMouth:

REPEATED CONCERN

Step 4 — Commitment Change

John:

> "Actually, I don't think I'll have enough time to handle deployment."

WiseMouth:

COMMITMENT CHANGED

Show the previous and current statements side by side.

Step 5 — Unanswered Question

Sarah:

> "Who's going to handle the migration?"

Nobody answers.

WiseMouth:

UNANSWERED QUESTION

Step 6 — Final Report

WISEMOUTH REPORT

5 Commitments
1 Commitment Changed
4 Uncertainty Signals
2 Hedging Signals
2 Repeated Concerns
1 Unresolved Disagreement

22. Hackathon Success Criteria

The MVP succeeds if judges can:

- 1 Open WiseMouth.
- 2 Start a conversation.
- 3 Speak naturally into the microphone.
- 4 See realtime transcription.
- 5 See conversational signals appear.
- 6 See evidence attached to signals.
- 7 See a commitment detected.
- 8 See that commitment change later.
- 9 See an unanswered question.
- 10 End the conversation.
- 11 Review the timeline.
- 12 Understand the product in under one minute.

23. What NOT to Build

Explicitly out of scope:

- Zoom integration
- Microsoft Teams integration
- Google Meet integration
- Mobile app
- Browser extension
- Facial emotion recognition
- Voice emotion classification
- Lie detection
- Personality profiling
- Psychological profiling

- Custom model training
- Fine-tuning
- Complex RAG
- Vector database
- Redis
- Kafka
- Kubernetes
- Microservices
- Terraform
- Complex authentication
- Billing
- Multi-tenant organizations

The MVP architecture should remain:

```

Next.js
|
FastAPI
|
AssemblyAI + LLM + PostgreSQL

```

24. Free-First Philosophy

Every technology should be evaluated in this order:

- 1 Open source
- 2 Free tier
- 3 Hackathon allowance
- 4 Existing infrastructure
- 5 Paid service only if absolutely necessary

Avoid introducing a paid dependency simply because it is convenient.

The core product should remain demonstrable even if an optional integration disappears.

25. The Pitch

Do not pitch WiseMouth as:

> "An AI meeting transcription tool."

Instead:

> **"A transcript tells you what people said. WiseMouth identifies the signals that emerge across the conversation."**

Then demonstrate:

> "John said he would handle deployment."

Later:

> "John said he probably won't have time."

WiseMouth connects those two moments.

Then:

> **"That's the difference. We're not just recording conversations. We're understanding how they evolve."**

26. Product Vision

The hackathon MVP focuses on meetings and project conversations.

The longer-term WiseMouth platform could support:

- Team meetings
- Client calls
- Sales conversations
- Interviews
- Classroom discussions
- Research discussions
- Architecture reviews
- Project planning
- Customer support
- Brainstorming

The underlying platform remains:

Conversation
|
v
Observable Signals

|
v
Conversation State
|
v
Temporal Relationships
|
v
Actionable Insight

27. Final Technology Decision

FREE / OPEN SOURCE

- Next.js
- TypeScript
- Tailwind
- shadcn/ui
- Lucide
- Recharts
- Python
- FastAPI
- Pydantic
- SQLAlchemy
- PostgreSQL
- Docker
- GitHub

HACKATHON / FREE ALLOWANCE

- AssemblyAI Realtime STT
- Groq
- Gemini fallback

FREE HOSTING TARGET

- Vercel
- Render or Oracle Cloud
- Free PostgreSQL provider or self-hosted PostgreSQL

OPTIONAL EMERGENCY FALLBACK

- Ollama + local model

28. Final Architecture Principle

AssemblyAI hears it.

Rules detect candidates.

The LLM interprets them.

Conversation state remembers them.

Temporal intelligence connects them.

WiseMouth makes them visible.

WiseMouth

See what the conversation doesn't explicitly say.

Meeting Environment: Lightweight Built-In Online Meeting

WiseMouth will **not depend on Google Meet for the 48-hour MVP**. Instead, we will build a very lightweight meeting-room interface that provides a realistic online meeting environment for demonstrating the core WiseMouth intelligence.

This is **not intended to compete with Google Meet or become a full video-conferencing platform**. The meeting UI is simply the controlled environment in which WiseMouth can capture a live conversation and demonstrate its analysis.

What the Built-In Meeting Includes

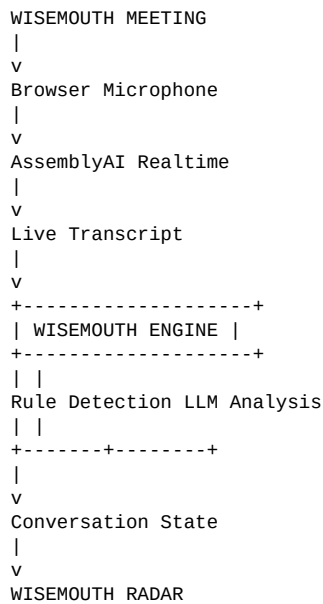
- Start / join a meeting
- Participant tiles
- Microphone mute/unmute
- Optional camera preview
- Live speaker/conversation area

- Live conversation/transcript display
- Leave/end meeting control
- WiseMouth radar alongside or within the meeting experience

Explicitly Out of Scope

- Screen sharing
- Meeting scheduling/calendar
- Chat system
- Recording infrastructure
- Background replacement
- Advanced noise suppression
- Authentication and organization management
- Full WebRTC conferencing infrastructure
- Building a Google Meet competitor

Demo Architecture



Demo Modes

Mode A — Live Meeting (primary demo)

The presenter starts the built-in WiseMouth meeting and speaks naturally. Audio is sent through the realtime transcription pipeline, signals are detected, and the radar updates live.

Mode B — Controlled Demo Conversation (fallback)

A deterministic demo conversation can be triggered when live audio, network connectivity, or transcription becomes unreliable during judging. The conversation contains predefined lines designed to demonstrate uncertainty, commitments, repeated concerns, unanswered questions, disagreements, and commitment changes.

Example:

- 1 Sarah: "I'm not sure we're ready for Friday." → Uncertainty
- 2 John: "I'll handle the deployment." → Commitment
- 3 Michael raises the authentication concern. → Concern
- 4 John later says: "I don't think I'll have enough time." → Commitment Change
- 5 A question is raised and receives no answer. → Unanswered Question

The fallback should still feed the **same WiseMouth signal engine and UI**, rather than displaying a fake pre-rendered result.

Product Positioning

The product is **not "an AI plugin for Google Meet."**

The core positioning is:

> **WiseMouth is a real-time intelligence layer for online conversations.**

The built-in meeting environment proves the core technology while keeping the architecture independent of a specific conferencing provider. Future integrations could connect the same intelligence engine to Google Meet, Zoom, Microsoft Teams, or other meeting platforms.

Recommended Demo Layout

```
+-----+
| WISEMOUTH MEETING | CONVERSATION RADAR |
| | |
| Sarah John Mike | ■ Uncertainty 2 |
| [video] [video] [video] | ■ Hedging 1 |
| | ■ Commitments 3 |
| Sarah: I'm not sure we're | ■ Commitment Changes 1 |
| ready for Friday... | ■ Repeated Concerns 2 |
| | ■ Unanswered Questions 1 |
| John: I'll handle the | |
| deployment. | LIVE SIGNAL |
| | ■ COMMITMENT CHANGED |
| ■ Mute ■ Camera Leave | Earlier: "I'll handle..." |
| | Later: "I don't think..." |
+-----+
```

48-Hour Scope Rule

The meeting interface is a **supporting component**. If time becomes constrained, prioritize:

- 1 Reliable microphone/audio input

- 2 AssemblyAI realtime transcription
- 3 Signal detection
- 4 Conversation state / temporal intelligence
- 5 Radar and evidence UI
- 6 Meeting-room visual polish
- 7 Optional multiplayer/video features

Do not sacrifice the WiseMouth intelligence engine to build conferencing features.